

Градиентный спуск

Паточенко Евгений

НИУ ВШЭ

План занятия

- Задача оптимизации
- Градиентный спуск
- Модификации градиентного спуска

Задача оптимизации

Определение

Обучение — это по сути процесс подбора весов или параметров модели так, чтобы *оптимизировать* (в большинстве случаев — минимизировать) функцию потерь.

Задача оптимизации

Определение

Обучение — это по сути процесс подбора весов или параметров модели так, чтобы *оптимизировать* (в большинстве случаев — минимизировать) функцию потерь.

Оптимизация — это задача нахождения экстремума (минимума или максимума) целевой функции в некоторой области конечномерного векторного пространства, ограниченной набором линейных и/или нелинейных равенств и/или неравенств.

Задача оптимизации

Постановка задачи

Среди элементов x , образующих множества X , найти такой элемент x^* , который доставляет минимальное (максимальное) значение $f(x^*)$ заданной функции $f(x)$.

Задача оптимизации

Постановка задачи

Среди элементов x , образующих множества X , найти такой элемент x^* , который доставляет минимальное (максимальное) значение $f(x^*)$ заданной функции $f(x)$.

В простейшем случае линейной регрессии:

Элементы x , образующие множества X — веса модели

Функция $f(x)$ — среднеквадратичная ошибка

Задача оптимизации

Постановка задачи

Среди элементов x , образующих множества X , найти такой элемент x^* , который доставляет минимальное (максимальное) значение $f(x^*)$ заданной функции $f(x)$

В простейшем случае линейной регрессии:

Элементы x , образующие множества X — веса модели

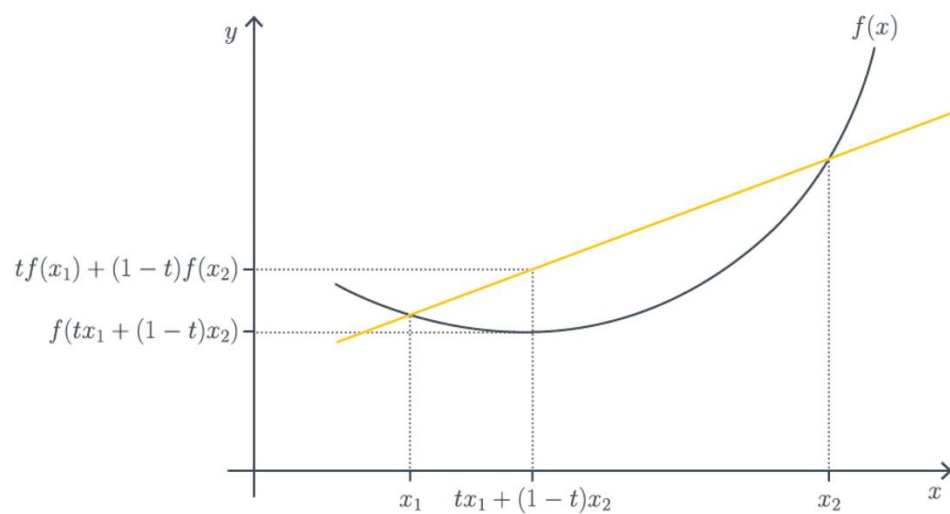
Функция $f(x)$ — среднеквадратичная ошибка

Почему оптимизация среднеквадратичной ошибки — простейший случай?

Задача оптимизации

Оптимизация MSE

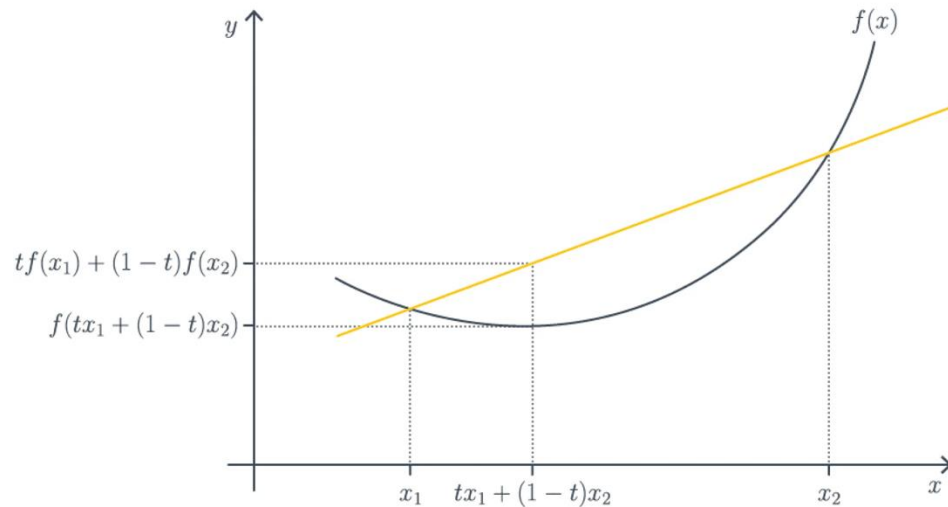
Функция среднеквадратичной ошибки —
выпуклая



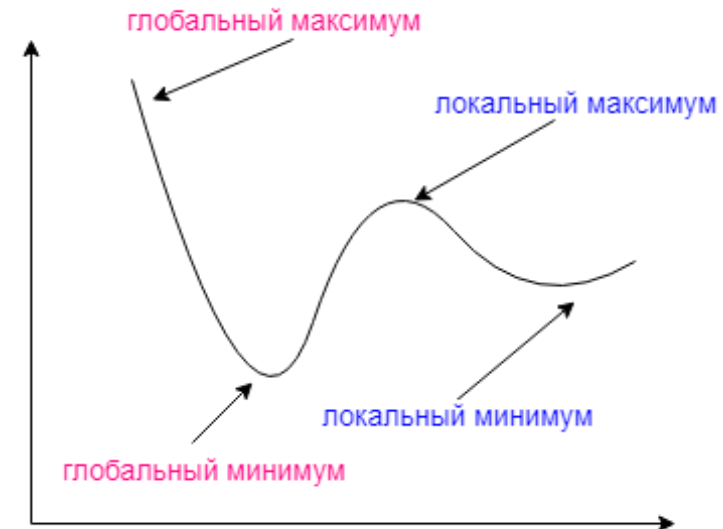
Задача оптимизации

Оптимизация MSE

Функция среднеквадратичной ошибки —
выпуклая



Для нас это означает, что у такой функции
локальный и глобальный минимумы совпадают
(в отличие от случая на картинке ниже)



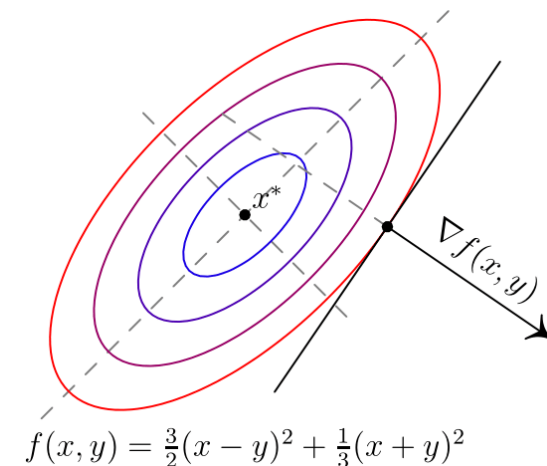
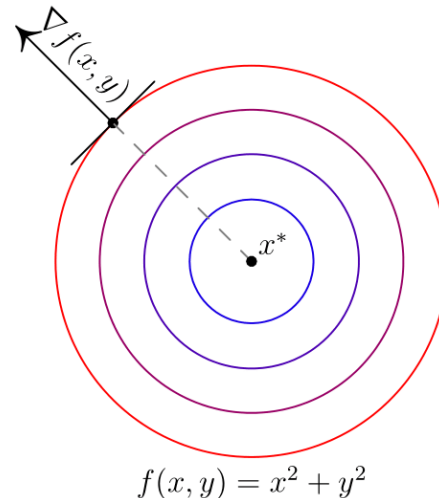
Задача оптимизации

Поиск минимума

Градиент — это вектор, в направлении которого функция быстрее всего растет

Антиградиент (вектор, противоположный градиенту) — вектор, в направлении которого функция быстрее всего убывает, при этом в общем случае он *не указывает точно на точку минимума функции*

Вектор градиента обозначают *grad* или ∇



Задача оптимизации

Градиент

Если f — функция n переменных $x_1, \dots, x_n$, ее градиентом называется n -мерный вектор $\left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n}\right)$, компоненты которого равны частным производным f по всем ее аргументам

Для одномерного пространства градиентом функции $f(x)$ будет $\frac{\partial f}{\partial x}$

Градиентный спуск

Определение

Градиентный спуск — метод оптимизации первого порядка для нахождения локального экстремума (минимума или максимума) функции с помощью движения вдоль градиента

Методы оптимизации:

- прямые — требуют только вычислений целевой функции в точках приближений
- первого порядка — требуют вычисления первых частных производных функции
- второго порядка — требуют вычисления вторых частных производных, то есть гессиана целевой функции

Градиентный спуск

Алгоритм

- Выбираем x_0 — начальную точку градиентного спуска.
- Каждую следующую точку выбираем по формуле:

$$x_{k+1} = x_k - \alpha \nabla f(x_k),$$

где α — это размер шага (learning rate)

- Проверяем критерий останова. Если не выполнен, повторяем предыдущий шаг.
Выполнен — останавливаемся

Градиентный спуск

Алгоритм

При обучении модели мы подбираем веса. Соответственно, формула

$$x_{k+1} = x_k - \alpha \nabla f(x_k),$$

Может быть записана так:

$$\beta_{k+1} = \beta_k - \alpha \nabla f(\beta_k)$$

То есть на каждом шаге для нахождения новой точки мы *обновляем веса*

Градиентный спуск

Инициализация весов

Градиентный спуск

Инициализация весов

- $\beta_k = 0, k = 1, \dots, n$

Градиентный спуск

Инициализация весов

- $\beta_k = 0, k = 1, \dots, n$
- Небольшие случайные значения: $\beta_k := \text{random}(-\varepsilon, \varepsilon)$ из некоторого распределения

Градиентный спуск

Инициализация весов

- $\beta_k = 0, k = 1, \dots, n$
- Небольшие случайные значения: $\beta_k := \text{random}(-\varepsilon, \varepsilon)$ из некоторого распределения
- Обучение по небольшой случайной подвыборке объектов

Градиентный спуск

Инициализация весов

- $\beta_k = 0, k = 1, \dots, n$
- Небольшие случайные значения: $\beta_k := \text{random}(-\varepsilon, \varepsilon)$ из некоторого распределения
- Обучение по небольшой случайной подвыборке объектов
- Мультистарт — многократный запуск из разных случайных начальных приближений и выбор лучшего решения

Градиентный спуск

Выбор градиентного шага (скорость обучения, learning rate)

α — это гиперпараметр, который мы подбираем и который влияет на качество и скорость обучения. При неудачном выборе метод может либо слишком медленно сходиться, либо пропускать минимум

Градиентный спуск

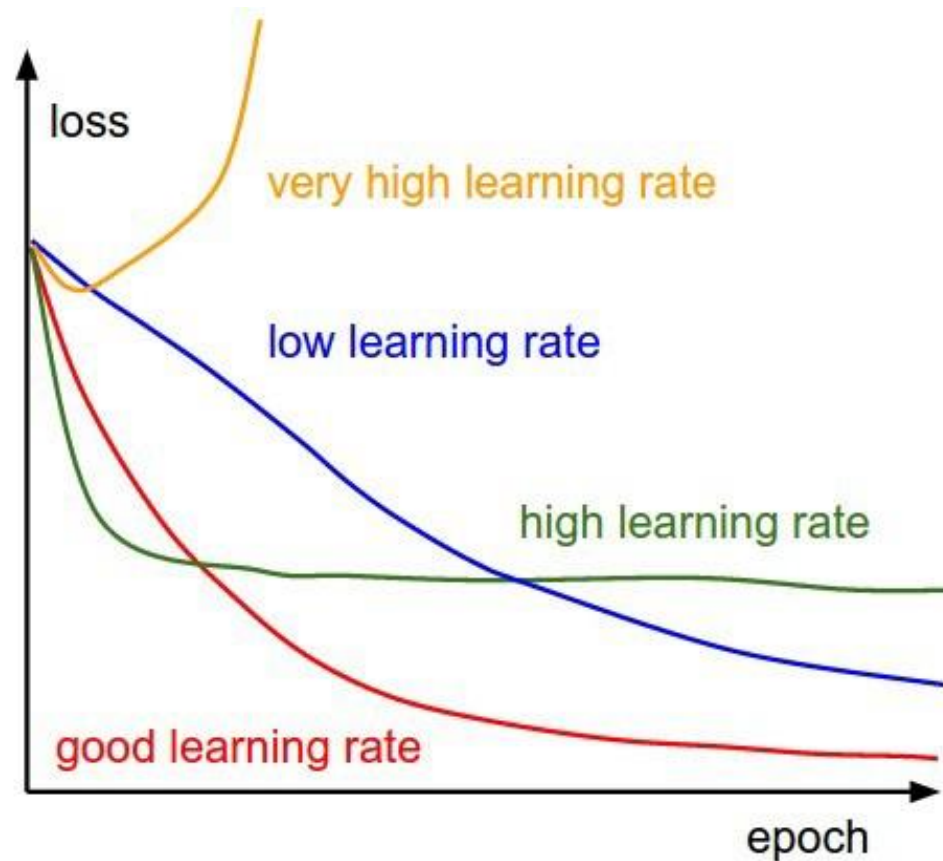
Выбор градиентного шага (скорость обучения, learning rate)

α — это гиперпараметр, который мы подбираем и который влияет на качество и скорость обучения. При неудачном выборе метод может либо слишком медленно сходиться, либо пропускать минимум

- Если хотим, чтобы получаемая точка минимума была как можно ближе к точному значению, выбираем как можно меньшую α
- Если важнее скорость и хотим уменьшить количество шагов, выбираем как можно большую α

Градиентный спуск

Выбор градиентного шага (скорость обучения, learning rate)



Градиентный спуск

Критерии останова

Градиентный спуск

Критерии останова

- Остановка через заранее заданное число шагов

Градиентный спуск

Критерии останова

- Остановка через заранее заданное число шагов
- Остановка на основе изменения функционала $|Q(\beta^{k+1}) - Q(\beta^k)| < \varepsilon$

Градиентный спуск

Критерии останова

- Остановка через заранее заданное число шагов
- Остановка на основе изменения функционала $|Q(\beta^{k+1}) - Q(\beta^k)| < \varepsilon$
- Остановка через изменение весов $\|\beta^{k+1} - \beta^k\| < \varepsilon$

Градиентный спуск

Недостаток градиентного спуска

Градиентный спуск

Недостатки градиентного спуска

- На каждом шаге мы вычисляем целую матрицу производных — по каждому весу от каждого объекта. Это затратно и по времени, и по памяти.

Градиентный спуск

Недостатки градиентного спуска

- На каждом шаге мы вычисляем целую матрицу производных — по каждому весу от каждого объекта. Это затратно и по времени, и по памяти.
- Может застрять в локальном минимуме

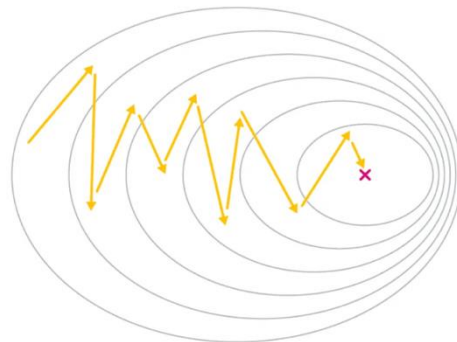
Модификации градиентного спуска

Стохастический градиентный спуск (SGD)

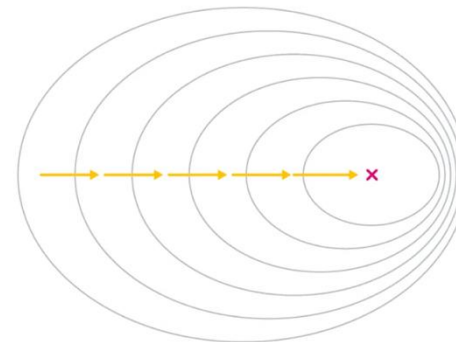
На каждом шаге выбираем один случайный (отсюда — «стохастический» в названии метода) объект и двигаемся в сторону антиградиента по нему

Такая модификация решает техническую проблему ресурсоемкости градиентного спуска, при этом вычисленный градиент, хоть и придет при правильно подобранном шаге обучения и начальной точке в точку минимума, может потребовать больше шагов, чтобы сойтись

Stochastic Gradient Descent



Gradient Descent



Модификации градиентного спуска

Mini-batch SGD

На каждом шаге выбираем подвыборку объектов (*batch size*, например, 32, 64 и т. д.) и двигаемся в сторону антиградиента функции потерь, вычисленной по батчу

